Universidad Nacional Autónoma de México — Facultad de Ingeniería
Fundamentos de Sistemas Embebidos — Lab 6b: Analog Data Acquisition
Rafael Ortega de la Paz | 420054085

1. Introduction

This lab is about reading an analog signal with a microcontroller and passing the result to a single-board computer. In my own words, the goal was to read an LM35 temperature sensor with the analog-to-digital converter (ADC) of a Raspberry Pi Pico W programmed in MicroPython, send the temperature over the I^{2}C bus to a Raspberry Pi 4, and have the Raspberry Pi log the readings, print them, plot their history and publish that history on a web page. The manual offers the same circuit with an Arduino UNO (version 6) or with the RP2040 (version 6b); I did version 6b because both the Pico W and the Raspberry Pi work with 3.3 V logic.

2. Background

Texas Instruments (2017) describes the LM35 as a sensor whose output rises linearly by 10 mV/°C, with an accuracy of 0.5 °C at 25 °C and a supply current below 60 µA. Its output cannot go below its own ground pin, so the manual (Matamoros, 2025) lifts that pin with two 1N914 diodes in series, shunted by an 18 kΩ resistor, and takes the temperature from the difference $V_{OUT+} - V_{OUT-}$. As Sedra and Smith (2015) explain, the forward voltage of a silicon junction depends on its current and drifts by about -2 mV/°C, so the offset is not a fixed value and both nodes have to be digitized.

Kester (2005) points out that an N-bit converter splits its full scale into 2^N codes, so the full scale fixes the size of every step. The RP2040 has a 12-bit ADC with four inputs (GP26 to GP29) whose full scale is the voltage on ADC_VREF (Raspberry Pi Ltd, 2023a). On the Pico W this pin is tied to the 3.3 V rail through a 200 Ω resistor (Raspberry Pi Ltd, 2023b). The manual sets it with a 10 kΩ/12 kΩ divider from 5 V (2.727 V), but Horowitz and Hill (2015) show that a divider behaves as a source with Thévenin resistance $R_1 \parallel R_2$, here about 5.5 kΩ, which cannot pull down a node held by 200 Ω. In MicroPython, `read_u16()` returns the 12-bit result scaled to 0–65535 (Bell, 2022).

The NXP specification (2021) defines I^{2}C as a two-wire bus (SDA and SCL), open-drain with pull-ups, in which the controller generates the clock and addresses a target with 7 bits. Here the Raspberry Pi is the controller, with its own 1.8 kΩ pull-ups, and the Pico is the target at `0x0A`. Since MicroPython only offers I^{2}C as a controller, the manual provides `i2cslave.py`, which configures the RP2040 peripheral as a target by writing its registers and works by polling.

3. Development

Wiring. I placed the Pico W across the breadboard with the USB connector at column 1 and built the LM35, the two diodes and the 18 kΩ resistor in columns 30 to 40. V_{OUT+} went to GP27 (ADC1, pin 32) and V_{OUT-} to GP26 (ADC0, pin 31). The warning of the manual says that VCC goes to “VBUS (pin 38)”, but pin 38 of the Pico is GND and VBUS is pin 40 (Raspberry Pi Ltd, 2023b), which is also what the manual’s figure shows, so I used pin 40. Before powering it, the multimeter showed two errors: the LM35 was rotated 180°, which put 5 V on its ground pin, and the cathode end of the diodes was not connected to ground, so V_{OUT-} was floating. After fixing both, I measured 0.788 V for V_{OUT-} and 3.330 V for ADC_VREF. The first value is lower than the 1.2 V of the manual because the 18 kΩ resistor takes part of the 50 µA of the sensor and each diode drops only about 0.4 V at that current; it does not matter because only the difference is used. The second

confirms that the divider cannot set the reference on this board, so I kept only the 12 kΩ resistor and wrote the measured 3.330 V in the code, as the calibration note of the manual recommends. The 0.1 µF capacitor went between the supply rails.

Reading the LM35. After installing MicroPython for the Pico W from Thonny, I ran `pico-code-adc.py`. The example failed: it imported `ADC` but used `machine.ADC`, `setup()` created local variables and was never called, and `read_temp()` had no `return`. Its formula also divides by 40.96, which corresponds to 4096 codes, while `read_u16()` returns 65536, so every reading would be 16 times too high. I used

$$T\,[^\circ\mathrm{C}] = \frac{(N_+ - N_-)\,V_{REF}}{65536 \times 0.01\ \mathrm{V}/^\circ\mathrm{C}} = \frac{(N_+ - N_-)\,V_{REF}}{655.36}.$$

With $V_{REF} = 3.330$ V one step is 0.81 mV, about 0.08 °C. At rest the program showed 24.5 °C, with $V_{OUT+} \approx 0.797$ V and $V_{OUT-} \approx 0.551$ V; the difference of 0.245 V is exactly 10 mV/°C. Holding the sensor raised the reading to 32.8 °C. The ADC read V_{OUT-} lower than the multimeter, probably because that node has a high source impedance, but the error appears in both channels and cancels in the difference.

I²C. I saved `i2cslave.py` on the Pico unchanged. The example `pico-code-i2c.py` did not import `machine` and its `read_temp()` always returned 25.0, so I wrote `main.py`: it averages eight readings, packs the result with `ustruct.pack('<f', t)` and writes the 4 bytes when the controller asks for them. The Raspberry Pi had two settings left from earlier labs. The kiosk of Lab 5 started from `/etc/rc.local` and used GPIO2 and GPIO3, which are SDA and SCL, as buttons, so I renamed the file to `/etc/rc.local.p5`. The access point of Lab 4 used 192.168.1.254, the same address as the gateway of the router, which left the board without Internet; I turned it off with `nmcli con down AccessPoint`, enabled I²C with `raspi-config`, installed `i2c-tools` and `python3-matplotlib`, and lowered the bus to 10 kHz (`dtparam=i2c_arm_baudrate=10000`) to give the polling target time to answer. Because pins 2 and 4 of the header carry 5 V right next to SDA and SCL, I measured the loose jumpers before connecting them to the Pico: both read 3.303 V. `i2cdetect -y 1` then listed the Pico at `0a`.

Log. The example `raspberry-code-i2c.py` opened the log with `'w+'`, which empties the file each time, and wrote the tuple returned by `struct.unpack`. I changed it to append mode, took element `[0]` and skipped failed or out-of-range readings. `temp.log` then recorded one line per second, from 23.81 °C to 28.80 °C while the sensor was warmed.

Experiment 1 (`exp1_imprime_temperatura.py`). It prints the date, the temperature and the four raw bytes of each reading and keeps appending to the log. The bytes are an IEEE-754 single-precision float, least significant byte first: `[72, 1, 0, 66]` is 32.00 °C, and the last byte changes from 65 to 66 when the value crosses 32 because it holds the exponent. The run went from 23.51 °C at rest to 32.48 °C while I held the sensor and then cooled down gradually. A single 34.96 °C was noise from touching the leads, since the next reading was 28.81 °C. Every request returned four valid bytes.

Experiment 2 (`exp2_grafica_bitacora.py`). A thread keeps reading the Pico while the plot re-reads `temp.log`, converts each line into a date and a temperature and draws it with matplotlib. The Pi has no desktop, so the `--png` option saves the figure without a window and I copied it with `scp`. It showed 185 readings between 23.47 and 34.96 °C, with three warm-ups, each followed by a fast cooling to about 26 °C within a minute and

a slower one towards 24.5 °C. A straight segment between 22:38:50 and 22:40:40 joined two runs separated by a reboot, so I changed the script to break the line when more than 10 s pass between readings.

Experiment 3 (`exp3_servidor_web.py`). One thread logs and another runs an HTTP server on port 8080 with `ThreadingHTTPServer` from the standard library. Each request reads the log, groups the readings by minute (mean, minimum, maximum and count), draws the chart as SVG on the server and lists the last ten minutes. The page refreshes every 60 s and lets the user choose 15 min, 1 h, 6 h, 24 h or the whole log. I opened it from a phone at `http://192.168.1.84:8080`; complete minutes held 59 or 60 readings, which confirms the 1 Hz sampling, and the minimum–maximum band was wide while I handled the sensor and almost vanished once it settled at 23.9 °C.

4. Conclusions

The whole chain worked: the LM35 was read by the RP2040, the Pico sent the temperature as an I^{2}C target at `0x0A`, and the Raspberry Pi logged, printed, plotted and published it with a resolution of one minute. The differential measurement was what made the result reliable: V_{OUT-} was far from 1.2 V and the ADC and the multimeter did not agree on it, but the difference still gave 10 mV/°C. Accuracy depends mainly on V_{REF}, because the temperature is proportional to it; writing the theoretical 2.727 V instead of the measured 3.330 V would have made every reading about 18 % too low.

The difficulties came from the material and from the reused hardware. The manual named a ground pin as VBUS, used a constant meant for 12-bit counts, and its examples did not run or kept only the last line of the log. The kiosk of Lab 5 occupied the I^{2}C pins and the access point of Lab 4 took the router's address. Measuring before powering the circuit avoided damaging the LM35 and putting 5 V on the I^{2}C lines.

References

Bell, C. (2022). *Beginning MicroPython with the Raspberry Pi Pico*. Apress.

Horowitz, P., and Hill, W. (2015). *The Art of Electronics*, 3rd ed. Cambridge University Press.

Kester, W. (Ed.) (2005). *The Data Conversion Handbook*. Newnes / Analog Devices.

Matamoros de Maria y Campos, J. M. (2025). *Práctica 6b: Lectura de datos analógicos usando el RP2040 y la Raspberry Pi*. Fundamentos de Sistemas Embebidos, UNAM.

NXP Semiconductors (2021). *UM10204: I^{2}C-bus specification and user manual*, Rev. 7.0.

Raspberry Pi Ltd (2023a). *RP2040 Datasheet*.

Raspberry Pi Ltd (2023b). *Raspberry Pi Pico W Datasheet*.

Sedra, A. S., and Smith, K. C. (2015). *Microelectronic Circuits*, 7th ed. Oxford University Press.

Texas Instruments (2017). *LM35 Precision Centigrade Temperature Sensors*, SNIS159H.